Assignment 3. Cloud Computing and Microservices Resilience

AITU Digital Student Services Platform

Astana IT University | Fault Tolerance | Academic Year 2026-2027

Context note: All operational data in this assignment are synthetic educational data. They are designed to represent a realistic Kazakhstan/Astana university context and do not describe actual AITU infrastructure, incidents, or internal systems.

Scenario

Design a resilient cloud-native architecture for a synthetic Astana IT University digital student-services platform. The platform supports timetable lookup, course information, registration requests, and student authentication.

Architecture: Users, Load Balancer, API Gateway, Auth / Course / Registration, Database.

Given requirements

Parameter	Requirement
Students/users	10,000
Peak requests	500 req/s
Availability target	99.9%
Maximum RTO	15 min
Maximum RPO	5 min
Maximum API latency	500 ms
Auth replicas	2
Course replicas	2
Registration replicas	2
Database nodes	2

Synthetic component availability

Component	Availability
Load Balancer	99.95%
API Gateway	99.90%
Auth instance	99.50%
Course instance	99.50%
Registration instance	99.50%
Database node	99.70%

Part A. Architecture Analysis (5 points)

Component	Failure scenario	Effect on student service	Resilience mechanism
Load Balancer	LB node down or wrong health check config	No request reaches the platform, full outage	Active and standby LB with a shared virtual IP, health checks to every gateway
API Gateway	Process crash, bad route config, overload	All APIs fail even if the services behind are healthy	At least 2 gateway replicas behind the LB, readiness probe, rate limit, config changes by rolling update
Auth	Pod crash, signing key store unreachable	New logins fail, students with a valid token keep working	2 replicas, short-lived signed tokens checked at the gateway with cached public keys
Course	Pod crash, slow queries	Timetable and course pages fail or load slowly	2 replicas, 500 ms timeout, gateway cache for course data (old data is acceptable here)
Registration	Pod crash, database write error, repeated submit	Students cannot register, a retry could register twice	2 replicas on 2 nodes, readiness tied to the database, UNIQUE(student, course) so a retry is safe, PDB
Database	Primary node down, disk full, bad data written	Registration and course data unavailable, writes can be lost	Primary and standby with streaming replication, backup every 5 min on another node, restore drill

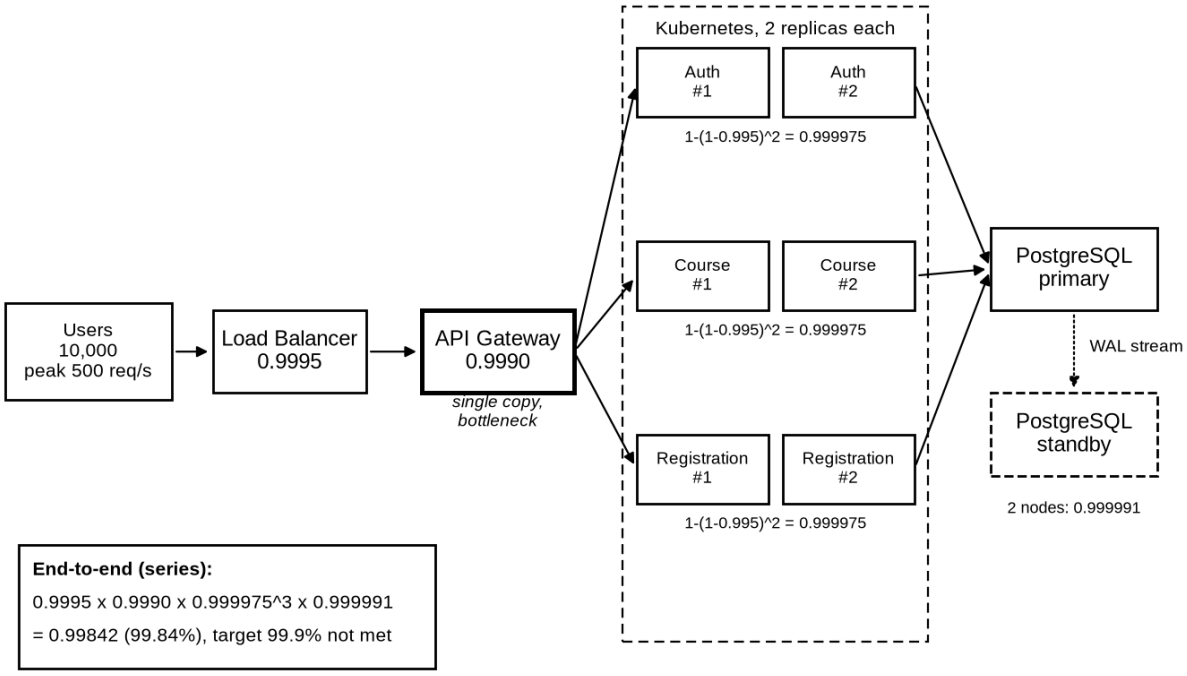


Figure 1. My architecture with the availability of every block. Only the Registration Service and PostgreSQL were deployed and tested in minikube. The standby node exists only in the design.

Part B. Availability Calculation (7 points)

For two independent replicas I use R_parallel = 1 - (1 - R1)(1 - R2). Blocks in the request path are in series, so their availabilities are multiplied. The numbers are produced by scripts/availability.py.

Subsystem	Configuration	Availability
Auth	2 replicas: 1 - (1 - 0.995)(1 - 0.995) = 1 - 0.000025	0.999975 (99.997%)
Course	2 replicas: 1 - (0.005)(0.005)	0.999975 (99.997%)
Registration	2 replicas: 1 - (0.005)(0.005)	0.999975 (99.997%)
Database	2 nodes: 1 - (1 - 0.997)(1 - 0.997) = 1 - 0.000009	0.999991 (99.999%)
End-to-end	Series: 0.9995 x 0.9990 x 0.999975 x 0.999975 x 0.999975 x 0.999991	0.998417 (99.842%)

1. Does the design meet 99.9%? No. The path gives 99.842%, which is 13.9 hours of downtime per year, while 99.9% allows 8.76 hours.
2. Which component is the bottleneck? The API Gateway. It is a single copy with 99.90%, so it brings 0.100% of unavailability, 63% of the total. The Load Balancer is second with 0.050% (32%). All replicated blocks together add less than 0.01%.
3. What redundancy change would have the largest effect? A second API Gateway replica. It raises the path to 99.942% and meets the target. A second Load Balancer alone gives only 99.892%, and a third replica of every service only 99.849%.

Change (one at a time)	End-to-end availability	Meets 99.9%
Given design	99.842%	No
Second API gateway	99.942%	Yes
Second load balancer	99.892%	No
Third replica of Auth, Course, Registration	99.849%	No
Third database node	99.843%	No
Second gateway and second load balancer	99.991%	Yes

The formula assumes independent failures and instant failover. Two replicas on the same node fail together, so in the YAML I spread them over two nodes. The measured failover in Part D took 0 to 2 s, which fits the assumption well for pod failures but not for the database, where the outage lasted as long as the database was down.

Part C. Kubernetes Resilience Design (7 points)

The Registration Service is a small Go program (registration/main.go) with PostgreSQL. The YAML below is k8s/registration.yaml from the repository. It has 2 replicas, a readiness probe, a liveness probe, resource requests and limits, a rolling update, automatic restart and service discovery through a ClusterIP Service. The database password in k8s/postgres.yaml is a demo value, and no real AITU endpoints or credentials are used.

```
# Registration Service of the synthetic AITU student services platform.
# Synthetic configuration only: no real AITU endpoints, credentials or infrastructure.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: registration
  namespace: asg3
  labels:
    app: registration
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
```

```
      app: registration
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # start one new pod first
      maxUnavailable: 0    # never go below 2 ready pods during an update
  template:
    metadata:
      labels:
        app: registration
    spec:
      restartPolicy: Always
      terminationGracePeriodSeconds: 20
      # spread the two replicas over the two nodes, but still schedule if one node is gone
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: registration
      containers:
        - name: registration
          image: registration:asg3
          imagePullPolicy: Never
          ports:
            - name: http
              containerPort: 8080
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: registration-db
                  key: url
          startupProbe:
            httpGet: {path: /health/live, port: http}
            periodSeconds: 1
            failureThreshold: 30
          livenessProbe:
            # process check only, a database outage must not restart the pods
            httpGet: {path: /health/live, port: http}
            periodSeconds: 5
            timeoutSeconds: 1
            failureThreshold: 3
          readinessProbe:
            # pod gets traffic only when PostgreSQL answers within 300 ms
            httpGet: {path: /health/ready, port: http}
            periodSeconds: 2
            timeoutSeconds: 1
            failureThreshold: 2
            successThreshold: 1
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 128Mi
          securityContext:
            runAsNonRoot: true
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
---
# service discovery: the API gateway calls http://registration.asg3.svc.cluster.local
apiVersion: v1
kind: Service
metadata:
  name: registration
  namespace: asg3
spec:
  selector:
    app: registration
  ports:
    - name: http
      port: 80
      targetPort: http
---
# NodePort only for the test client that runs outside the cluster
apiVersion: v1
kind: Service
metadata:
  name: registration-test
  namespace: asg3
spec:
  type: NodePort
  selector:
    app: registration
  ports:
    - name: http
      port: 80
      targetPort: http
      nodePort: 30090
---
apiVersion: policy/v1
```

```
kind: PodDisruptionBudget
metadata:
  name: registration
  namespace: asg3
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: registration
```

Mechanism	Implemented?	Purpose
Replicas	Yes. replicas: 2, spread over the two nodes with topologySpreadConstraints	One pod or one node can fail and the other pod keeps serving
Liveness probe	Yes. GET /health/live every 5 s, restart after 3 failures	Restart a hung process. It does not check the database, so a database outage does not cause a restart loop
Readiness probe	Yes. GET /health/ready every 2 s, 2 failures, checks PostgreSQL with a 300 ms ping	Take a pod out of the Service when it cannot serve, put it back when it can
Resource limits	Yes. Requests 100m CPU and 64Mi, limits 500m CPU and 128Mi	The scheduler knows the pod size, and one pod cannot take the whole node
Rolling update	Yes. maxSurge 1, maxUnavailable 0, 4 s drain after SIGTERM	New version without dropping below 2 ready pods
Automatic restart	Yes. restartPolicy Always, the Deployment replaces deleted pods	A crashed container restarts, a lost pod is recreated
Service discovery	Yes. Service registration, DNS name registration.asg3.svc.cluster.local	The gateway finds ready pods by name, not by IP
Disruption budget	Yes. PodDisruptionBudget minAvailable 1	A node drain cannot remove both pods at once

Part D. Failure Simulation (6 points)

Experiment	Injected failure	Detection time	Recovery time	User impact
1	One Registration replica (pod killed, grace period 0)	Kubernetes 0.19 s (replacement pod created). No request failed	Replacement ready after 1.83 s, the service was never down	0 of 976 requests failed, 0 of 680 writes lost
2	All Registration replicas (both pods killed)	Kubernetes 0.23 s, first failed or slow request 1.15 s	Both pods ready 1.55 s, requests OK again 2.25 s after the kill	1 failed and 4 slow (up to 1.05 s) of 912 (0.55%), 0 writes lost
3	Database unavailable (PostgreSQL scaled to 0 for 20 s)	First error 0.12 s, readiness removed both pods after 3.54 s	Pods ready 3.32 s and requests OK 4.17 s after the database returned. Whole outage 24.2 s	116 of 831 requests failed (14.08%), 0 acknowledged writes lost, no failed write was saved

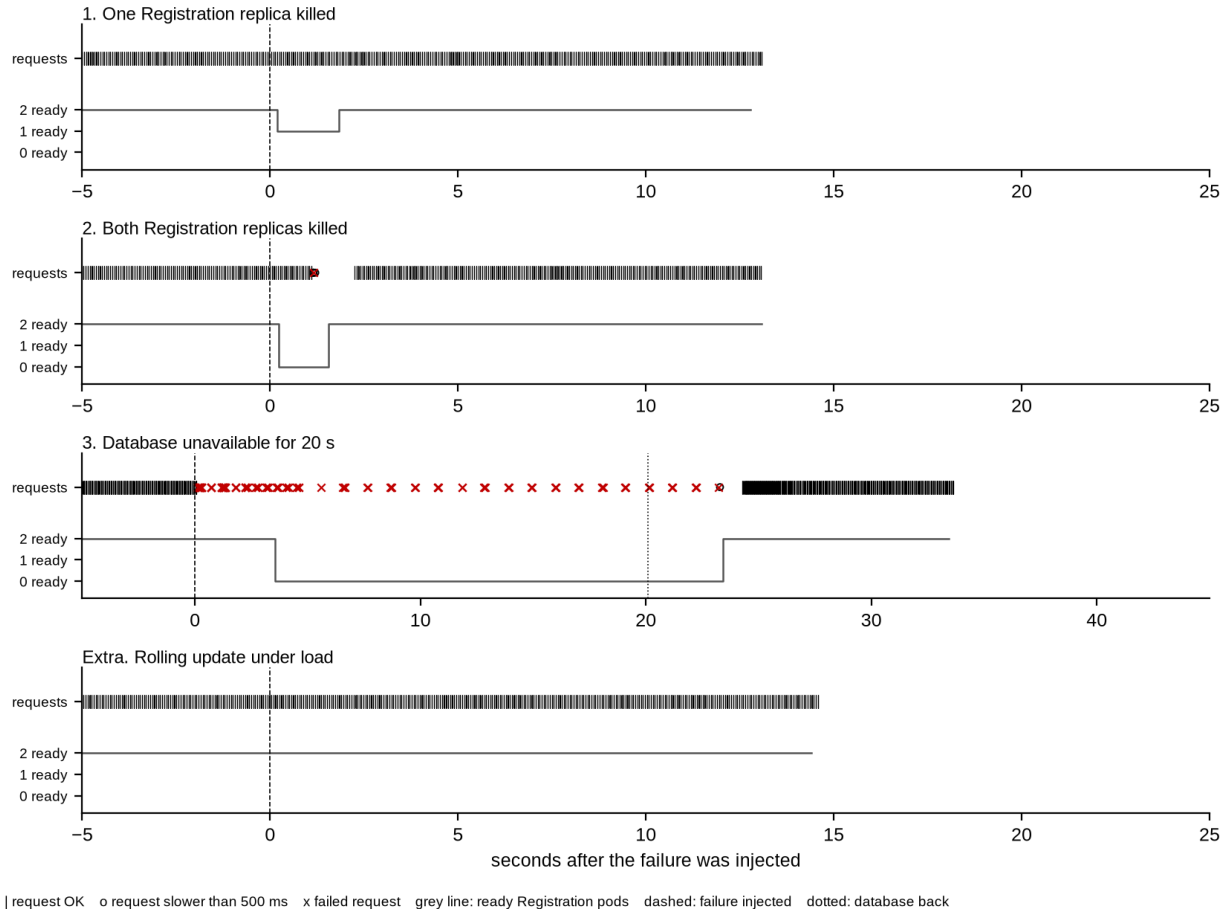


Figure 2. Request results and ready Registration pods during the experiments, drawn from results/.json.*

I measured everything with scripts/experiment.py. It runs 4 client threads against the NodePort Service. Every thread sends a request, waits 50 ms and repeats. Half of the requests are writes with a new student ID, half are reads. A request fails on an HTTP error or after a 2 s timeout, and it is slow when it takes more than 500 ms. A second thread reads the pod list every 0.25 s. The failure is injected 5 s after the start with kubectl. Detection time is the time from injection to the first reaction (Kubernetes) or the first bad request (user). Recovery time is the time until both replicas are ready and requests succeed again. At the end the script reads the database and checks that every write that got HTTP 200 is stored and that no failed write was stored.

In experiment 3 the first 62 failures were HTTP 503 from the service itself: the pods were still in the Service but could not reach PostgreSQL. After the readiness probe failed, the Service had no endpoints and 54 requests were refused at connection. Users got a fast error and no hanging requests. As an extra test I ran a rolling update under the same load: the rollout took 4.1 s and 0 of 1096 requests failed.

Part E. RPO/RTO and Disaster Recovery (5 points)

Mechanism	Frequency	Expected RPO	Recovery time	Purpose
Database backup	pg_dump every 5 min (CronJob), stored on the other node, last 12 kept	At most 5 min	Measured: restore job 3.2 s for 2,617 rows	Bring data back after loss, corruption or a bad change

Mechanism	Frequency	Expected RPO	Recovery time	Purpose
Replica	Continuous streaming replication to the standby node (design)	Seconds with async replication, 0 with sync	1 to 2 min to promote the standby	Survive a database node failure without a restore
Configuration backup	All YAML in git with a commit for every change, etcd snapshot daily	0 for the YAML in git	Measured: PostgreSQL recreated from YAML in 3.1 s	Rebuild Kubernetes objects after a lost namespace or cluster
Full recovery	Drill after every big change, at least once per semester	At most 5 min (last dump)	Measured: 7.3 s from deleted volume to working API, 15 min target	Prove that backup, YAML and steps really work together

High availability and disaster recovery solve different problems. High availability keeps the service running through expected component failures without a human, within seconds: in Part D one or two pods died and the service was back in 0 to 2.3 s. It works by having more copies of the same thing, so it also copies mistakes. A DELETE, a bad migration or ransomware is replicated to the standby within seconds. Disaster recovery brings back a correct earlier state after an event that high availability cannot absorb, such as a lost volume or corrupted data. It is measured by RPO and RTO and needs a procedure. In my drill (scripts/dr_drill.sh) I deleted the PostgreSQL StatefulSet together with its volume. Replicas could not help here, but the backup could: the service answered again after 7.3 s with all 2,617 rows.

My first drill attempt failed. The restore started while the new database was still initialising, and the scheduled backup ran against the empty new database and moved latest.dump to an empty dump. The fixed procedure suspends the backup CronJob, restores a named dump file and waits for a real database connection.

Submission

Architecture diagram: Figure 1

Availability calculations: Part B and results/availability.txt

Kubernetes YAML: Part C and k8s/ in the repository

Failure experiment results: Part D and results/

RPO/RTO plan: Part E and results/dr-drill.log

Oral defense

STUDENT TECHNICAL WORK REPORT / SUBMISSION RECORD

Assignment 3. Cloud Computing and Microservices Resilience

Purpose: This form is a formal submission record. The student must complete every applicable field and attach evidence of their own work. Short generic explanations are not sufficient.

1. Student and Submission Information

Required field	Student entry
Student full name	Alibek Asset
Student ID	255175
Group / cohort	CSE-2503
Instructor	Azamat Serek
Date of submission	12.10.2026
Individual variant / task code assigned by instructor	
Git repository / project link	https://github.com/Gontafi/university-stuff/tree/FTR-assignment-03
Attached code / file names	registration/main.go, registration/Dockerfile, k8s/registration.yaml, k8s/postgres.yaml, k8s/backup.yaml, scripts/experiment.py, scripts/availability.py, scripts/dr_drill.sh, results/ (raw samples, events, drill logs), this report

2. Work Period and Task Record

Item	Student entry
Work start date/time	12.10.2026, 14:20
Work completion date/time	12.10.2026, 14:35 (code and results committed), report on the same day
Main task completed	Parts A to E: component analysis, availability calculation, Kubernetes YAML for the Registration Service, three failure simulations plus a rolling update test, backup and disaster recovery drill
Tools / technologies used	Go 1.26 with pgx, PostgreSQL 17, Docker, minikube v1.37 with two nodes, Kubernetes 1.37, Python 3 standard library, matplotlib for the figures
Main technical output produced	Working Registration Service on two nodes, YAML, raw request samples and pod snapshots for 4 experiments, Kubernetes events, availability tables, backup CronJob and a timed restore drill

3. Report Structure to Complete

For each section below, write the requested evidence in your own words. Include calculations, diagrams, code references, screenshots, logs, or experimental evidence where applicable.

1. Architecture Analysis

Analyze the failure scenarios for each component and explain the resilience mechanism selected for your design.

Required content	Student response / evidence
Summary	Every block in the path can stop the whole service. The LB and the gateway are single copies in the given design, so I add a second gateway first. Services get 2 replicas on 2 nodes, the database a standby and a backup every 5 minutes.
Evidence / calculation / artifact reference	Part A table, Figure 1, k8s/ folder.
Student-specific decision or observation	For Course I accept old data from a gateway cache, for Registration I do not. Instead I make retries safe with UNIQUE(student, course).

2. Availability Calculation

Show the parallel-redundancy calculations and the end-to-end series calculation. Explain whether the target is met and identify the bottleneck.

Required content	Student response / evidence
Summary	Replicated services 0.999975 each, database 0.999991, end-to-end 99.842%. Target 99.9% not met. Bottleneck: API Gateway with 63% of the unavailability.
Evidence / calculation / artifact reference	Part B tables, scripts/availability.py, results/availability.txt.
Student-specific decision or observation	I calculated every single change. Only a second gateway reaches the target (99.942%). More service replicas add less than 0.01 percentage points.

3. Kubernetes Resilience Design

Include the Registration Service YAML or a clear excerpt/reference. Explain replicas, probes, resources, rolling update, restart, and service discovery.

Required content	Student response / evidence
Summary	2 replicas with topology spread, liveness, readiness and startup probes, requests and limits, rolling update with maxUnavailable 0, restartPolicy Always, ClusterIP Service and a PodDisruptionBudget.
Evidence / calculation / artifact reference	Part C YAML and table, k8s/registration.yaml, results/cluster-state.txt (pods on both nodes, endpoints).
Student-specific decision or observation	Readiness checks the database, liveness does not. A database outage then removes pods from traffic but never restarts them.

4. Failure Simulation

Document all three required simulations with detection time, recovery time, and user impact. Explain how each result was obtained.

Required content	Student response / evidence
Summary	One replica: 0 failed requests. Both replicas: 1 failed and 4 slow, back after 2.2 s. Database: 116 failed, outage 24.2 s for a 20 s database stop. No acknowledged write was lost in any run.
Evidence / calculation / artifact reference	Part D table and Figure 2, results/one-replica.json, all-replicas.json, database.json, rolling-update.json and the *-events.txt files.
Student-specific decision or observation	I also checked data: every write that got HTTP 200 was in the database, and no failed write was stored.

5. RPO/RTO and Disaster Recovery

Complete the RPO/RTO plan and explain why high availability and disaster recovery are different engineering objectives.

Required content	Student response / evidence
Summary	Backup every 5 minutes on the other node gives RPO at most 5 min. The full recovery drill took 7.3 s against a 15 min target. HA copies mistakes, DR restores a correct earlier state.
Evidence / calculation / artifact reference	Part E table, k8s/backup.yaml, scripts/dr_drill.sh, results/dr-drill.log, results/dr-drill-first-attempt.log.
Student-specific decision or observation	The recovery time is short only because the database is small (2,617 rows). The hard part was the procedure. The copy itself took seconds.

6. Final Architecture Decision

State the most important resilience change you made and justify it using availability, failure, or recovery evidence.

Required content	Student response / evidence
Summary	The most important change is a second API Gateway replica. It is the only single change that meets 99.9% (99.842% to 99.942%).
Evidence / calculation / artifact reference	Part B what-if table. Part D shows that a pair of replicas with probes hides a pod failure completely (0 failed requests).
Student-specific decision or observation	The same pattern I measured for Registration (2 replicas, readiness, rolling update) should be used for the gateway.

4. Individual Work and Anti-Plagiarism Record

Because the assignment uses a common educational scenario and dataset, the submitted report must contain verifiable evidence of the individual student's work. Copying another student's wording, calculations, architecture, code, logs, screenshots, or conclusions is not acceptable.

Use the instructor-assigned individual variant/task code.

Provide the exact YAML filename or repository commit used for the reported configuration.

Include evidence from your own simulation or test run.

Explain one Kubernetes setting you changed or tuned and why.

Be prepared during oral defense to explain what happens if both replicas fail.

Individual evidence	Student entry
My unique technical decision	Readiness depends on the database (300 ms ping), liveness does not, and the service fails readiness and waits 4 s after SIGTERM before it stops. With maxUnavailable 0 this gave 0 failed requests during the rolling update.
My own implementation / calculation evidence	Go service, three YAML files, experiment and drill scripts written and run by me on 12.10.2026. All numbers in Parts B, D and E come from the files in results/.

Individual evidence	Student entry
Exact file, code, commit, notebook, or log reference	Branch FTR-assignment-03, commit 84b5f84. k8s/registration.yaml, results/*.json, results/dr-drill.log.
Kubernetes setting I tuned and why	Readiness periodSeconds 2 and failureThreshold 2 instead of the default 10 s and 3, so a pod without a database leaves the Service in about 4 s. Measured: 3.5 s in experiment 3.
One problem I personally encountered and how I solved it	The first recovery drill failed. The restore job connected while PostgreSQL was still running initdb on the new volume (the pod was already Ready) and got connection refused. Meanwhile the CronJob made a dump of the empty database and latest.dump pointed to it. I suspend the CronJob during recovery, restore a named file and wait for a real connection. The second drill passed.
One limitation of my own solution	Only Registration and PostgreSQL are deployed. The standby database, gateway, Auth and Course exist only in the design. Everything runs on one laptop and the image was already on the nodes, so the restart times are better than in a real cluster.
If both replicas fail	The Service has no endpoints, requests fail until the Deployment starts new pods. Measured: back after 2.2 s, 1 failed and 4 slow requests.

5. Evidence Checklist

Evidence	Included	Where
Completed required tables and calculations	Yes	Parts A to E
Architecture / RBD / FTA / security diagram	Yes	Figure 1
Source code or configuration files	Yes	Part C, branch FTR-assignment-03
Execution logs / experiment results	Yes	results/ in the repository
Screenshots or other reproducible evidence	Yes	Figure 2 from raw data, README commands
Links to code, datasets, notebooks, or artifacts	Yes	Section 1 of this form
Before/after comparison	Yes	Part B what-if table, Part E drill
Final technical conclusion	Yes	Section 3.6 of this form

6. Student Declaration

I confirm that this report describes my own work. I have not copied another student's report, code, calculations, diagrams, logs, screenshots, or conclusions. Any external source or reused material has been identified. I understand that I may be asked to reproduce or explain any part of this submission during the oral defense.

Role	Name	Signature	Date
Student / Executor	Alibek Asset		12.10.2026
Instructor / Accepted by	Azamat Serek		

7. Instructor Acceptance Record

Check	Status / comment
Report structure completed	Yes No Comment:
Evidence is sufficient	Yes No Comment:
Individual contribution is identifiable	Yes No Comment:
Oral defense completed	Yes No Comment:
Final status	Accepted Revision required